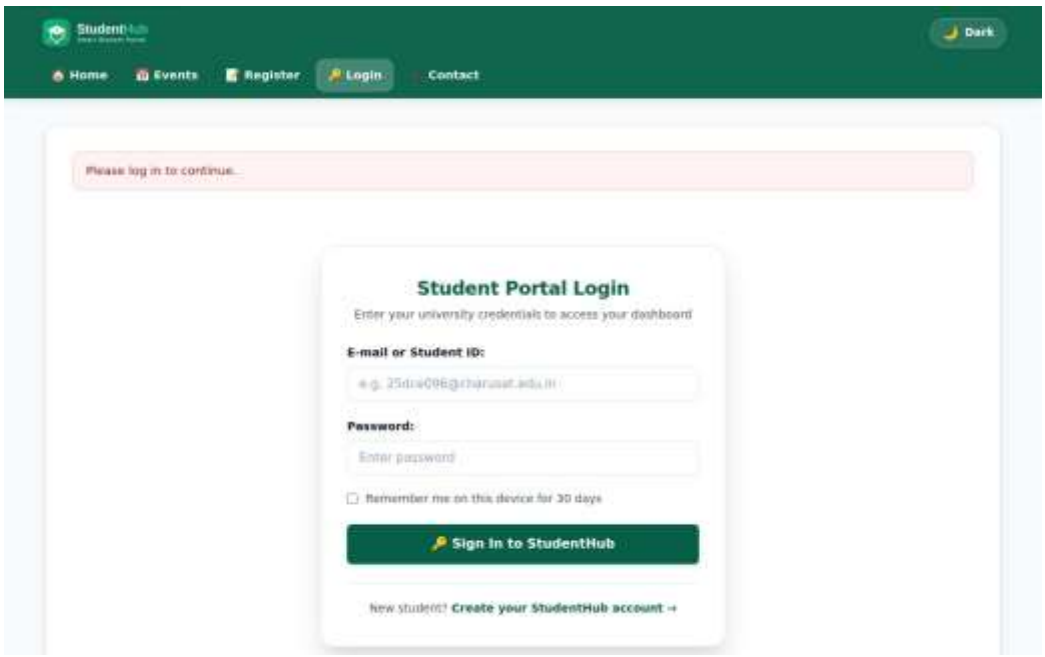
PRACTICAL-10

AIM: Build secure login and logout with PHP sessions: verify passwords with password_verify(), regenerate the session ID after login, restrict pages by role, enforce an idle timeout, and offer an optional remember-me token.

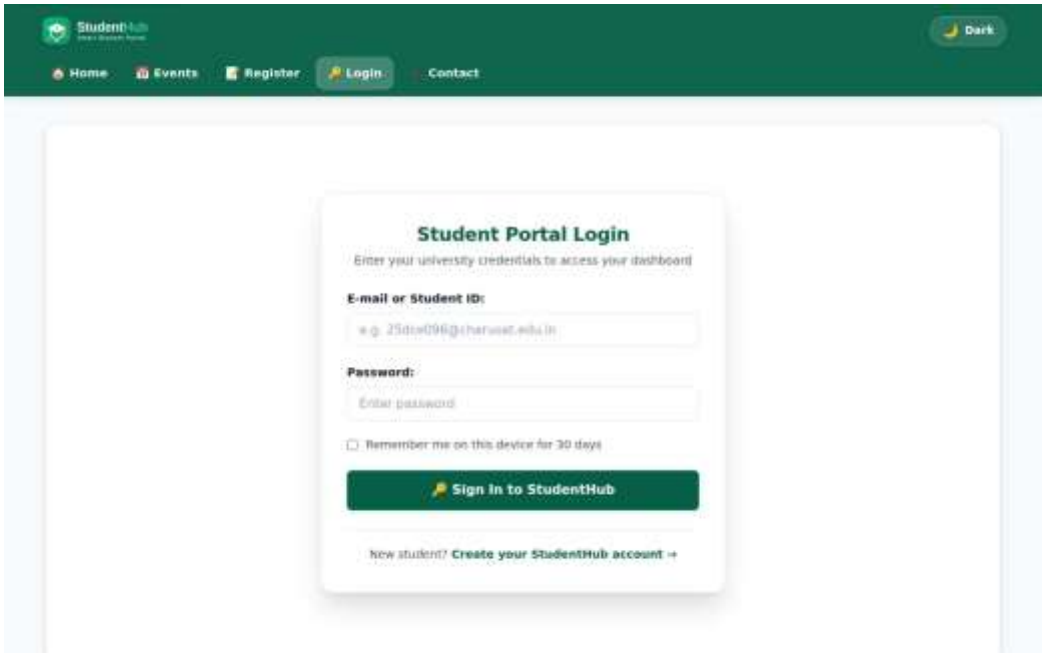
SOURCE CODE :

https://github.com/rachhadiyabrind-del/WDF_B/tree/main/Practical(1-4)(Studenthub)/Practical-10

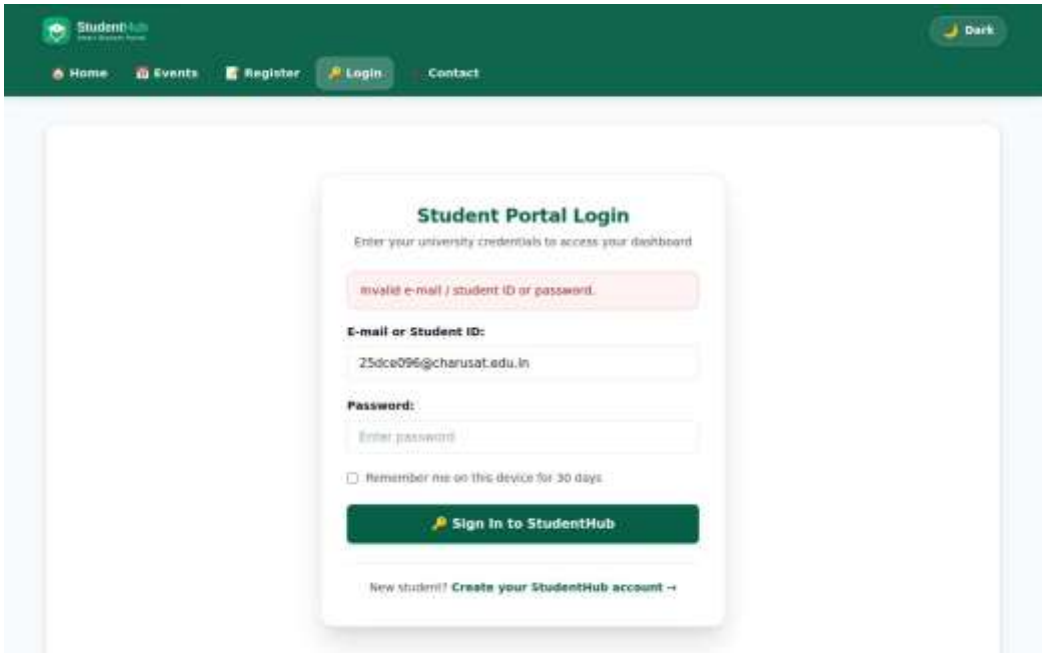
PREVIEW:



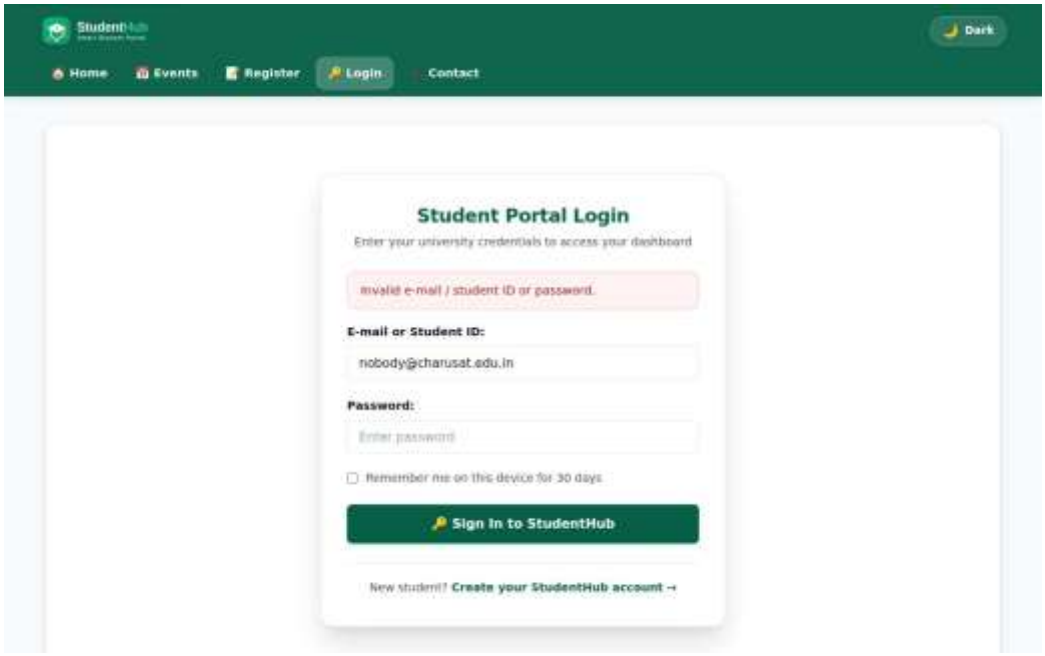
Guest opens the dashboard and is redirected to login



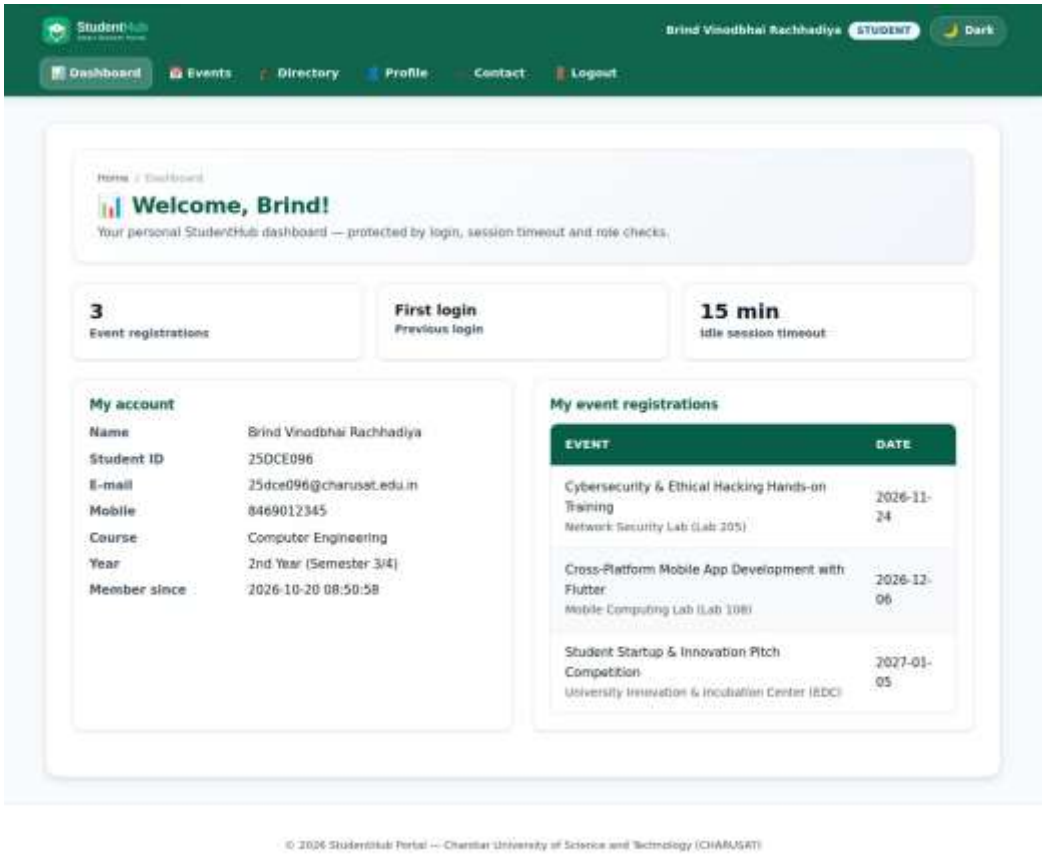
Login page



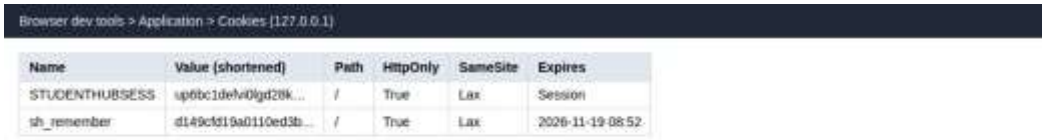
Wrong password - generic error message



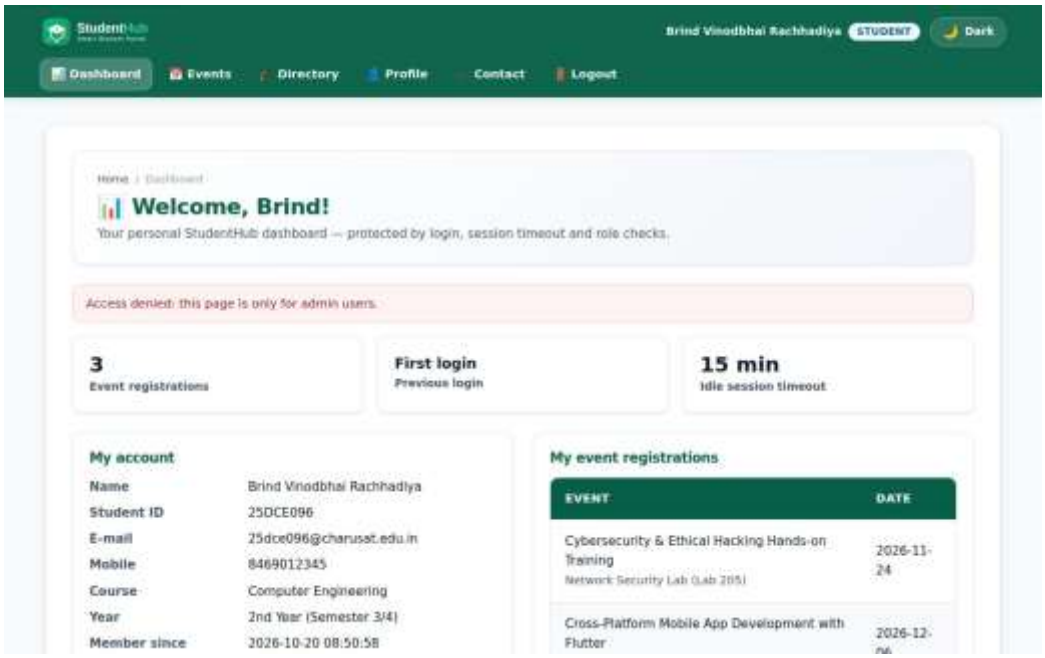
Unknown user - same generic message (no account enumeration)



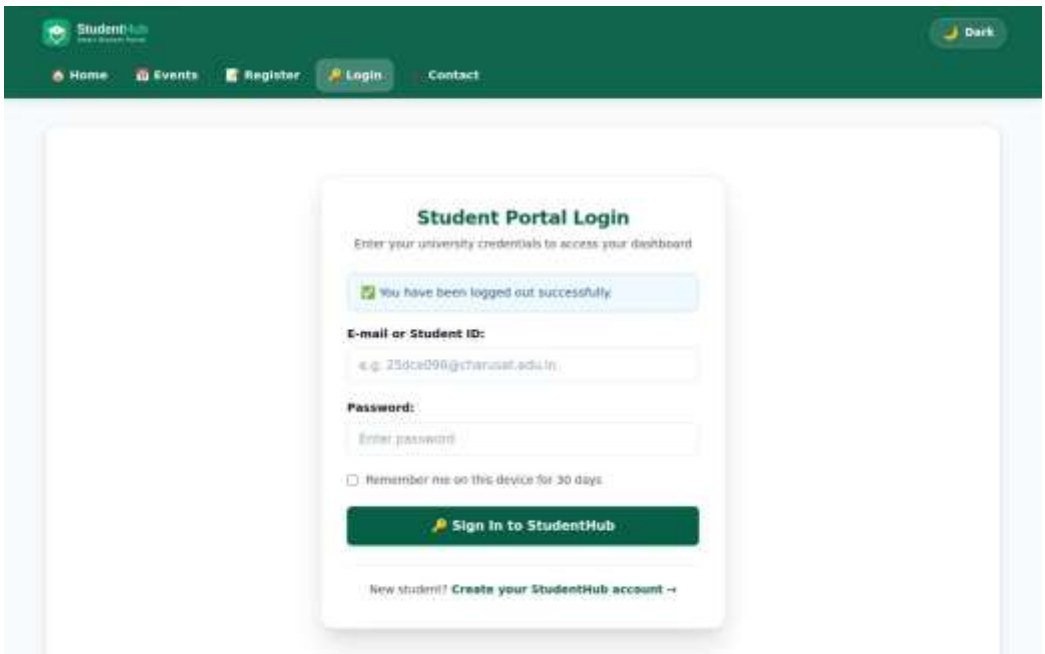
Student login - redirected to the student dashboard



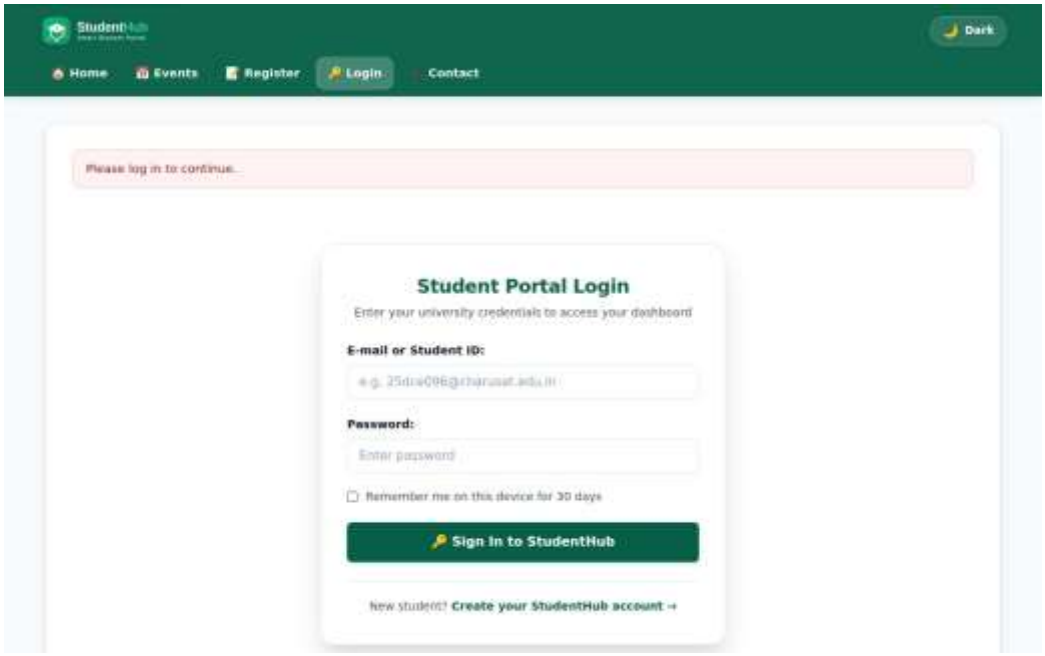
Session and remember-me cookies (HttpOnly, SameSite=Lax)



Student opens an admin page - access denied



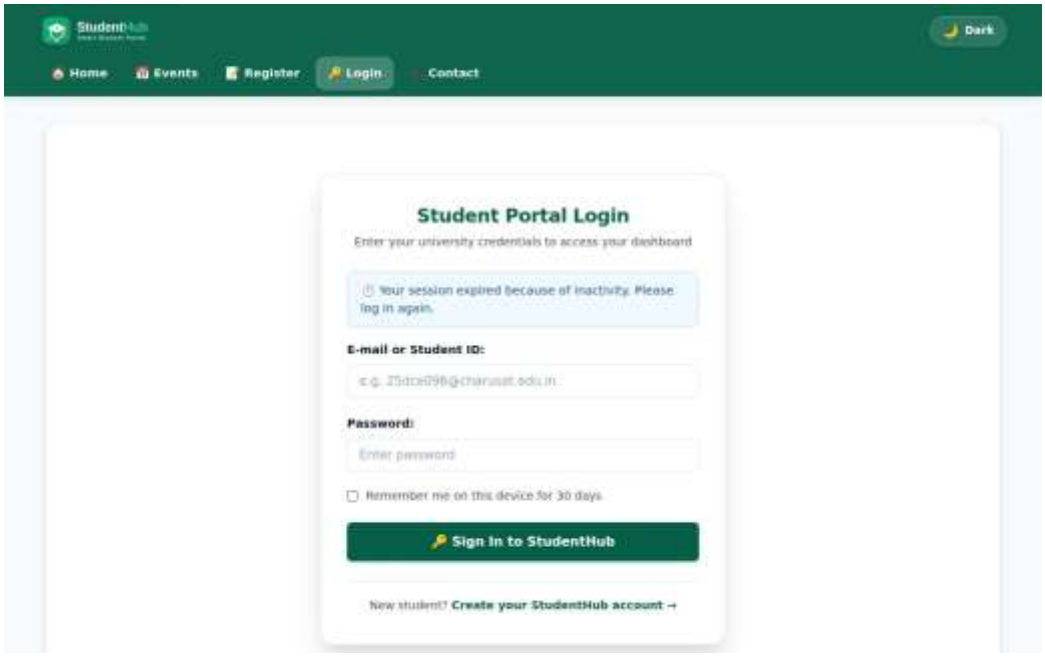
Logout - session ended



Dashboard after logout - access blocked



Admin login - redirected to the admin dashboard



Idle session expired - user must log in again

TEST CASES :

#	Input / action	Expected result	Result
1	Open the dashboard without logging in	Redirected to the login page with a "please log in" message	Pass
2	Login with the correct e-mail and wrong password	Generic "invalid credentials" error	Pass
3	Login with an unknown e-mail	Same generic message as a wrong password (no account enumeration)	Pass
4	Login as a student with the correct password	Session created; redirected to the student dashboard; last login shown	Pass
5	Tick "remember me" and log in	sh_remember cookie set with HttpOnly and SameSite=Lax; token stored as a hash	Pass
6	Student opens an admin-only page	"Access denied" message; redirected to the student home; access_denied logged	Pass
7	Logout, then press Back and open the dashboard	Session and remember-me token removed; login page shown	Pass
8	Login as admin	Redirected to the admin dashboard	Pass
9	Stay idle longer than the timeout, then click a protected link	Session destroyed; redirected to login with the timeout message; session_timeout logged	Pass

KEY QUESTIONS :

1. Are sessions started and destroyed securely?

Yes. After a successful password check, login_user() calls session_regenerate_id(true), which issues a new session ID so an attacker who knew the old ID cannot reuse it (session fixation). Logout calls destroy_session(): it clears $_SESSION, expires the session cookie, and calls session_destroy(). logout_user() also deletes the remember-me row from the database and expires that cookie.

```
session_regenerate_id(true);
```

2. Is password_verify() used correctly?

Yes. The typed password is compared with the stored bcrypt hash, not with any stored text. The same statement checks whether the account is active. If PHP's default hashing has changed, password_needs_rehash() detects the old hash and the code stores a new one after a successful login.

```
if ($user && password_verify($password, $user['password_hash'])) {
```

3. Is unauthorized dashboard access blocked?

Yes. Every protected page calls require_login() or require_role(), which check for $_SESSION['user']. If it is missing, the visitor is redirected to the login page with a flash message. Attempts to open a page for the wrong role are redirected and written to audit_logs as access_denied.

4. Is session timeout implemented?

Yes. SESSION_TIMEOUT is set in includes/config.php to 900 seconds (15 minutes), and it can be changed with the SH_TIMEOUT environment variable. Each request compares the current time with $_SESSION['last_activity']; if the gap is too long, the session is destroyed, the event is logged as session_timeout, and the user goes to login?timeout=1. Remember-me login is turned off after a timeout, so the user must type the password again.

5. Are admin and student users redirected to appropriate dashboards?

Yes. After login, the code checks the role read from the database: admins go to the admin dashboard and students go to the student dashboard. require_role('admin') enforces the same rule on each admin page, so a student who types an admin URL is sent back to the student home. The remember-me token uses a random selector and a random validator; only a SHA-256 hash of the validator is stored, the token is rotated on each use, and it expires after 30 days.